Marina Osiechko

March 10, 2026

Foundations Of Python Programming

Assignment 07

GitHub URL

Python Programming Tools and Techniques

Introduction

This paper describes the steps I followed to complete Assignment Seven in the ***Introduction to Programming with Python*** course. This module focused on core programming concepts, including classes, objects, constructors, attributes, and properties. Through this assignment, I strengthened my understanding of how classes differ from functions and statements, and how objects store and manage data. The module also introduced Git and GitHub, clarifying how each tool supports version control and streamlines collaborative development.

Task 1: Data Layer - Defining Data Constants and Variables

The assignment began with importing the ***json module*** and defining the program's constants and variables. The constant **MENU** stored the list of user choices using a multi-line string created with triple quotes. As required, constants were written in uppercase and remained unchanged throughout the program.

Variables were written in lowercase and were designed to store user input or program data. The variable ***menu_choice*** was initialized as empty string, and **students** was defined as data type list, that functions as a two-dimensional table of student data. This section represents the Data Layer, which is responsible for storing and organizing the program's state.

Figure 1 shows the code used to define constants and variables:

```
import json

# Define the Data Constants
MENU: str = '''
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------
'''
FILE_NAME: str = "Enrollments.json"

# Define the Data Variables
students: list = []  # a table of student data
menu_choice: str  # Hold the choice made by the user.
```

Figure 1: Defining the Data Constants and Variables

Task 2: Creating the Person Class

In this task, I created a Person class, which inherits code from Python's base object class. I began by creating the constructor (__init__), which initializes the first_name and last_name properties. I then implemented getter and setter methods for both properties using the @property decorator. These methods ensure that the data is properly formatted and validated before being stored.

At the bottom of this class, I overrode the __str__() method so that a Person object returns readable person information instead of a default memory address. This makes the class more user-friendly and easier to display in output.

Figure 2 shows the code used to create the Person class:

```
# TODO Create a Person Class (Done)
# This newly created Person class inherits code from Python's object class.
class Person:
    """
    A class representing person data.

    Properties:
    - first_name (str): The student's first name.
    - last_name (str): The student's last name.

    ChangeLog:
    - MOsiechko, 3.8.2026: Created class Person.
    """

    # TODO Add first_name and last_name properties to the constructor (Done)
    def __init__(self, first_name: str = '', last_name: str = ''):
        self.first_name = first_name
        self.last_name = last_name

    # TODO Create a getter and setter for the first_name property (Done)
    @property   #(Use this decorator for the getter or accessor)
    def first_name(self):
        return self.__first_name.title()    # formatting code

    @first_name.setter
    def first_name(self, value: str):
        if value.isalpha() or value == "":   # is character or empty string
            self.__first_name = value
        else:
            raise ValueError("The first name should not contain numbers.")

    # TODO Create a getter and setter for the last_name property (Done)
    @property  # (Use this decorator for the getter or accessor)
    def last_name(self):
        return self.__last_name.title()  # formatting code

    @last_name.setter
    def last_name(self, value: str):
        if value.isalpha() or value == "":  # is character or empty string
            self.__last_name = value
        else:
            raise ValueError("The last name should not contain numbers.")

    # TODO Override the __str__() method to return Person data (Done)
    def __str__(self):
        return f'{self.first_name},{self.last_name}'
```

Figure 2: Creating the Person Class

Task 3: Creating the Student (Person) Class

In this task, I created a Student class that inherits from the Person class. I began by calling the Person constructor and passing it the first_name and last_name values. Using the super() function allowed the Student class to reuse and extend the functionality already defined in the Person class.

Next, I added an assignment to the course_name property by using the course_name parameter provided to the constructor. I then created the getter and setter methods for the course_name attribute to ensure consistent access and storage of the data.

Finally, I overwrote the __str__ () method so that a Student object returns the formatted string containing the student's first_name, last_name, and a course_name. This makes the output more readable and user friendly.

Figure 3 shows the code used to create the Student (Person) class:

```
# TODO Create a Student class that inherits from the Person class (Done)
class Student(Person):
    """
        A collection data about students

        ChangeLog: (Who, When, What)
        Marina Osiechko, 3/8/2026,Created Class
        Marina Osiechko, 3/8/2026,Added properties and private attributes
        """

    # TODO call to the Person constructor and pass it the first_name and last_name data (Done)
    # super method connects first and last name to the Person class
    def __init__(self, first_name: str = '', last_name: str = '',
                 course_name: str = ''):
        super().__init__(first_name=student_first_name,
                         last_name=last_name)

        # TODO add an assignment to the course_name property using the course_name parameter
(Done)
        self.course_name = course_name

    # TODO add the getter for course_name (Done)
    @property
    def course_name(self):
        return self.__course_name

    # TODO add the setter for course_name (Done)
    @course_name.setter
    def course_name(self, value: str):
        self.__course_name = value

    # TODO Override the __str__() method to return the Student data (Done)
    def __str__(self):
        return f'{self.first_name},{self.course_name}'
```

Figure 3: Creating Class Student (Person)

Task 4: Processing Layer – Reading and Writing JSON Data

The Processing Layer, also known as the *Business Logic Layer*, implements the core functionality of the application. It is responsible for transforming data, enforcing program rules, and managing structured error handling. In this assignment, the Processing Layer handled reading student data from a JSON file, converting that data into Student objects, and writing updated data back to the file.

The FileProcessor class contains two static methods:

- ***read_data_from_file()***, which loads JSON data, converts each dictionary row into a Student object, and returns list of these objects using json.load().

- ***write_data_to_file()***, which converts Students objects back into dictionaries and writes them to the JSON file in a structured format using json.dump().

Both methods include try/except/finally blocks to ensure that errors – such as missing files, invalid JSON, or write conflicts – are handled gracefully. This structured approach keeps the program stable and prevents crashes caused by unexpected data or file access issues.

Figure 4 shows the code used to implement the Processing Layer:

```
# Processing ------------------------------------- #
class FileProcessor:
    """
    A collection of processing layer functions that work with JSON files

    ChangeLog: (Who, When, What)
    RRoot,1.1.2030,Created Class
    Marina Osiechko,3.8.2026,Modified Code
    """
    @staticmethod
    def read_data_from_file(file_name: str, students: list):
        """ This function reads data from a JSON file and loads it into a list of dictionary rows
        then returns the list filled with student data.

        ChangeLog: (Who, When, What)
        RRoot,1.1.2030,Created function
        Marina Osiechko,3.8.2026,Converted dictionary data to Student data
        Marina Osiechko,3.9.2026,Converted Student objects into dictionaries

        :param file_name: string data with name of file to read from
        :return: list
        """
        file = None
        student_objects = []

        try:
            # Get a list of dictionary rows from the data file
            file = open(file_name, "r")
            json_students = json.load(file)  # the load function returns a list of dict rows.

            # Convert the list of dictionary rows into a list of Student objects
            # TODO replace this line of code to convert dictionary data to Student data (Done)
            # student objects = json students
            for row in json_students:    # Convert the list of dict rows into Student objects
                students.append(Student(first_name=row["FirstName"],
                                        last_name=row["LastName"],
                                        course_name=row["CourseName"]))
        except Exception as e:
            IO.output_error_messages(message="Error: There was a problem with reading the file.",
error=e)

        except json.JSONDecodeError as e:
            IO.output_error_messages(message="Error: The file contains invalid JSON.", error=e)

        finally:
            if file is not None and file.closed == False:
                file.close()

        return students

    @staticmethod
    def write_data_to_file(file_name: str, student_data: list):
        """ This function writes data to a JSON file with data from a list of dictionary rows

        ChangeLog: (Who, When, What)
        RRoot,1.1.2030,Created function
        Marina Osiechko,3.8.2026, Added code converting student objects to dictionaries

        :param file_name: string data with name of file to write to
        :param student_data: list of dictionary rows to be written to the file
```

```
    :return: None
    """
    file = None
    json_students = []

    try:
        # TODO Add code to convert Student objects into dictionaries (Done)
        for student in student_data:   # Convert List of Student obj to list of dict rows.
            student_json: dict \
            = {"FirstName": student.first_name,
               "LastName": student.last_name,
               "CourseName": student.course_name}
            json_students.append(student_json)

        file = open(file_name, "w")
        json.dump(json_students, file, indent=2)

        IO.output_student_courses(student_data=student_data)
    except Exception as e:
        message = "Error: There was a problem with writing to the file.\n"
        message += "Please check that the file is not open by another program."
        IO.output_error_messages(message=message, error=e)
    finally:
        if file is not None and file.closed == False:
            file.close()
```

Figure 4: Reading and Writing Data from and to JSON file

Task 5: Presentation Layer – Managing User Input and Output

The ***Presentation Layer,*** also known as the Input/Output Layer, manages how the user interacts with the program. It displays information, collets user input, validates information, and displays the results in a readable format. This layer ensures that the program communicates effectively with the user and guides them through each step of the process.

This layer includes the following functions:

- ***output_error_messages()*** – Displays custom error messages to help the user understand what went wrong.
- ***output_menu()*** – Shows the menu of available options.
- ***input_menu_choice()*** – Collects and validates the user's menu selection.
- ***input_student_data()*** - Prompts the user for first name, last name, and course name, validates the input, and creates a Student object instead of the dictionary object.
- ***output_student_courses()*** – Displays the stored student registration data to the user as coma-separated values, accessing Student object properties rather than dictionary keys.

Each function includes descriptive document strings and structured error handling to guide the user and prevent invalid input.

Figure 5 shows the code that manages user input and output:

```
# Presentation ------------------------------------- #
class IO:
    """
    A collection of presentation layer functions that manage user input and output

    ChangeLog: (Who, When, What)
    RRoot,1.1.2030,Created Class
    Marina Osiehcko,3.7.2026,Modified Code
    """

    @staticmethod
    def output_error_messages(message: str, error: Exception = None):
```

```python
    """ This function displays the custom error messages to the user

    ChangeLog: (Who, When, What)
    RRoot,1.3.2030,Created function

    :param message: string with message data to display
    :param error: Exception object with technical message to display
    :return: None
    """
    print(message, end="\n\n")
    if error is not None:
        print("-- Technical Error Message -- ")
        print(error, error.__doc__, type(error), sep='\n')

@staticmethod
def output_menu(menu: str):
    """ This function displays the menu of choices to the user

    ChangeLog: (Who, When, What)
    RRoot,1.1.2030,Created function

    :return: None
    """
    print()  # Adding extra space to make it look nicer.
    print(menu)
    print()  # Adding extra space to make it look nicer.

@staticmethod
def input_menu_choice():
    """ This function gets a menu choice from the user

    ChangeLog: (Who, When, What)
    RRoot,1.1.2030,Created function
    Marina Osiechko,3.8.2026,Added code to access Student object data instead of dict data

    :return: string with the users choice
    """
    choice = "0"
    try:
        choice = input("Enter your menu choice number: ")
        if choice not in ("1","2","3","4"):  # Note these are strings
            raise Exception("Please, choose only 1, 2, 3, or 4")
    except Exception as e:
        IO.output_error_messages(e.__str__())  # Not passing e to avoid the technical message

    return choice

@staticmethod
def output_student_courses(student_data: list):
    """ This function displays the student and course names to the user as coma-separated
    values.

    ChangeLog: (Who, When, What)
    RRoot,1.1.2030,Created function
    Marina Osiechko, 3.8.2026, Changed code to access Student object instead of dict data

    :param student_data: list of dictionary rows to be displayed
    :return: None
    """

    print("-" * 50)
    for student in student_data:

        # TODO Add code to access Student object data instead of dictionary data (Done)
        #print(f'Student {student["FirstName"]} '
        #      f'{student["LastName"]} is enrolled in {student["CourseName"]}')
        print(student)
    print("-" * 50)

@staticmethod
```

```python
    def input_student_data(student_data: list):
        """ This function gets the student's first name and last name,
          with a course name from the user

        ChangeLog: (Who, When, What)
        RRoot,1.1.2030,Created function
        Marina Osiechko,3.8.2026, Replaced code to use Student object instead of dict objects

        :param student_data: list of dictionary rows to be filled with input data
        :return: list
        """

        try:
            first_name = input("Enter the student's first name: ")
            if not first_name.isalpha():
                raise ValueError("The last name should not contain numbers.")
            last_name = input("Enter the student's last name: ")
            if not last_name.isalpha():
                raise ValueError("The last name should not contain numbers.")
            course_name = input("Please enter the name of the course: ")

            # TODO Replace this code to use a Student obj instead of a dictionary objects (Done)

            student= Student(first_name, last_name, course_name)

            student_data.append(student)
            print()
            print(f"You have registered {student.first_name} {student.last_name} for
{student.course_name}.")
        except ValueError as e:
            IO.output_error_messages(message="One of the values was the correct type of data!",
error=e)
        except Exception as e:
            IO.output_error_messages(message="Error: There was a problem with your entered
data.", error=e)
        return student_data
```

Figure 5: Managing User Input and Output

Task 6: Main Body of the Script – Calling the Functions

The main section of the program uses a ***while True*** loop to repeatedly display the menu and process the user's selection. The program uses ***if/elif*** statements to handle the four menu choices shown in Figure 1.

Figure 6 shows the code used to call the functions in the main body of the script:

```python
# Start of main body

# When the program starts, read the file data into a list of lists (table)
# Extract the data from the file
students = []
students = FileProcessor.read_data_from_file(file_name=FILE_NAME)
#FileProcessor.read_data_from_file(FILE_NAME, students)

# Present and Process the data
while (True):

    # Present the menu of choices
    IO.output_menu(menu=MENU)

    menu_choice = IO.input_menu_choice()

    # Input user data
    if menu_choice == "1":  # This will not work if it is an integer!
        students = IO.input_student_data(student_data=students)
        continue

    # Present the current data
```

```
    elif menu_choice == "2":
        IO.output_student_courses(students)
        continue

    # Save the data to a file
    elif menu_choice == "3":
        FileProcessor.write_data_to_file(file_name=FILE_NAME, student_data=students)
        continue

    # Stop the loop
    elif menu_choice == "4":
        break  # out of the loop
    else:
        print("Please only choose option 1, 2, or 3")

print("Program Ended")
```

Figure 6: Calling the Functions

Because the functions are defined inside classes, the main script calls them using the class name to indicate where each function is located. This reinforces the Separation of Concerns by keeping the main logic separate from the data, processing, and presentation layers.

- **Menu choice 1** prompts the user for input and adds the collected data to the ***students*** list.
- ***Menu choice 2*** displays the current data using a formatted message.
- ***Menu choice 3*** saves the data to the JSON file.
- ***Menu choice 4*** ends the program.

When the user enters a number outside the valid range, the program displays a custom message directed the to choose a valid option.

Task 7: Code Testing

I tested the program by running it and confirming that it accepted user input, displayed existing file contents, formatted data into comma-separated strings, and saved output to the JSON file. The program successfully handled multiple registrations and ran correctly in both PyCharm and the terminal. These tests demonstrated that the classes, file-processing functions, and input/output features worked together as intended.

Summary

In this module, I learned how to create classes and how to organize and manage data using object-oriented programming principles. I discovered that storing information as a list of Student objects provides greater control, improves data validation, and makes the program easier to understand and maintain. Overall, this assignment strengthened my understanding of classes, properties, constructors, and JSON file handling, while also reinforcing the importance of clean program structure and layered design.